Class Relationships and Reuse

C++ Interlude 4

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Inheritance Revisited

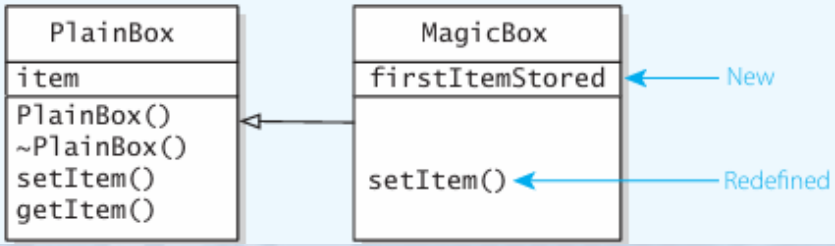
- Derived class inherits public/protected members of its base class
  - Do not inherit the constructors and destructor
  - Can add new members it defines
  - Can revise any inherited methods
- Inheritance enables the reuse of existing classes

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Multiple Inheritance

- Derived class can have more than one base class – multiple inheritance
- A method in a derived class redefines a nonvirtual method in the base class if the two methods have the same name and parameter declarations
- An instance of a derived class contains copies of inherited data members, but the code for inherited methods is not copied.
- A derived class cannot access the private members of the base class directly by name, even though they are inherited. Inheritance does not imply access.
- A derived class can use the base class's public methods to set or get the value of base item(s) indirectly.
- If a new method has the same name as an ancestor method— setItem, for example—instances of the new class will use the new method, while instances of the ancestor class will use the original method (early (static) binding).

FIGURE C4-3 The derived class `MagicBox` inherits members of the base class `PlainBox` and redefines and adds members



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Early versus Late Binding

```
// C++ Program to illustrate early (static) binding
#include<iostream>
using namespace std;

class Base
{
public:
   void show()
      {cout << " In Base \n"; }
};

class Derived : public Base
{
public:
   void show()
      {cout << "In Derived \n"; }
};

int main()
{
   Base* bp = new Derived;
   bp->show(); // RUN-TIME POLYMORPHISM
   delete bp;
   bp = nullptr;
   return 0;
}
```

This will execute

```
// C++ Program to illustrate late (dynamic) binding
#include<iostream>
using namespace std;

class Base
{
public:
   virtual void show()
      {cout << " In Base \n"; }
};

class Derived : public Base
{
public:
   void show()
      {cout << "In Derived \n"; }
};

int main()
{
   Base* bp = new Derived;
   bp->show(); // RUN-TIME POLYMORPHISM
   delete bp;
   bp = nullptr;
   return 0;
}
```

This will execute

https://www.geeksforgeeks.org/early-binding-late-binding-c/

Multiple Inheritance

- Derived class can have more than one base class – multiple inheritance
- A method in a derived class redefines a nonvirtual method in the base class if the two methods have the same name and parameter declarations

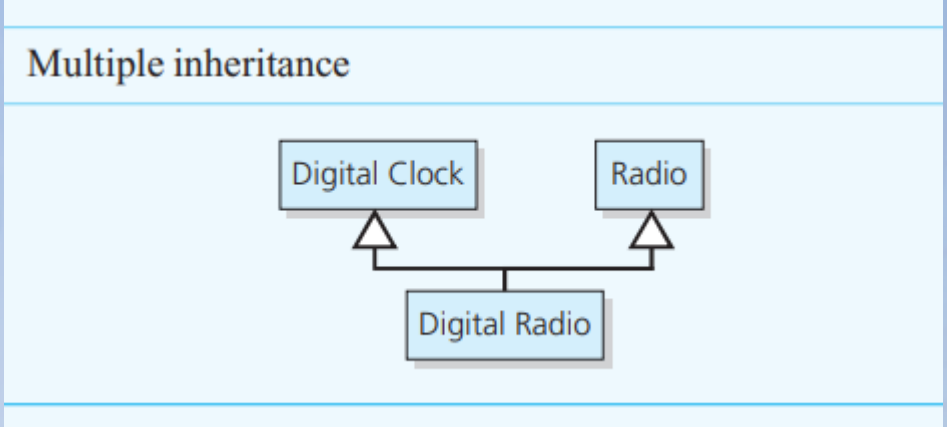
```
#include <iostream>

class DigitalClock {
public:
   void turnOn() {
      std::cout << " DigitalClock 's turnOn" << std::endl;
   }
};

class Radio {
public:
   void turnOn() {
      std::cout << " Radio's turnOn" << std::endl;
   }
};

class DigitalRadio : public DigitalClock, public Radio {
   // Inherits turnOn() from both DigitalClock and Radio
};

int main() {
 DigitalRadio obj;
   obj.turnOn(); // ERROR: which turnOn() to call?
}
```



ambiguity ERROR: which turnOn() to call?

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Multiple Inheritance

Ambiguity Resolution - Explicit qualification

```
#include <iostream>

class DigitalClock {
public:
   void turnOn() {
      std::cout << " DigitalClock 's turnOn" << std::endl;
   }
};

class Radio {
public:
   void turnOn() {
      std::cout << " Radio's turnOn" << std::endl;
   }
};

class DigitalRadio : public DigitalClock, public Radio {
   // Inherits turnOn() from both DigitalClock and Radio
};

int main() {
 DigitalRadio obj;
   obj.turnOn(); // ERROR: which turnOn() to call?
```

Ambiguity Resolution - Explicit qualification: specify which base class's turnOn method using the scope resolution operator (::)

```
obj.Radio::turnOn()
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Multiple Inheritance

```
#include <iostream>

class DigitalClock {
public:
  virtual void turnOn() {
      std::cout << " DigitalClock 's turnOn" << std::endl;
   }
};

class Radio {
public:
   void turnOn() {
      std::cout << " Radio's turnOn" << std::endl;
   }
};

class DigitalRadio : public DigitalClock, public Radio {
public:
  void turnOn() override {
      std::cout << " DigitalRadio turnOn" << std::endl;
   }
};

int main() {
 DigitalRadio obj;
   obj.turnOn(); // DigitalRadio’s turnOn will be called
```

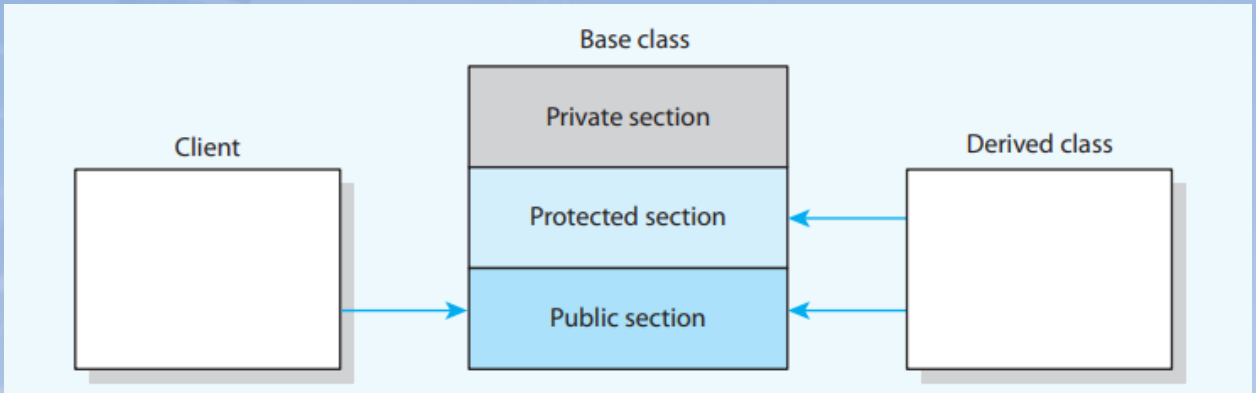
Ambiguity Resolution - Override the method in the derived class (virtual inheritance)

Ambiguity Resolution:override - You can explicitly define a turnOn method in the derived class DigitalRadio to resolve the ambiguity

1. Add virtual to one of the base classes
2. Redefine(override) the method in the derived class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Public, Private and Protected



- Public members can be used by anyone.
- Private members can be used only by methods of the class.
- Protected members can be used only by methods of the class or any derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Public, Private and Protected Inheritance

Inheritance Type	Public Members	Protected Members	Private Members
Public	Remain **public**	Become **protected**	**Not inherited**
Private	Become **private**	Become **private**	**Not inherited**
Protected	Become **protected**	Remain **protected**	**Not inherited**

- Public inheritance: Public and protected members of the base class remain, respectively, public and protected members of the derived class.
- Private inheritance: Public and protected members of the base class are private members of the derived class.
- Protected inheritance: Public and protected members of the base class are protected members of the derived class.
- In all cases, private members of a base class remain private to the base class and cannot be accessed by a derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Is-a and As-a Relationships

- Is-a relationships
  - Public inheritance implements an is-a relationship
  - In general, a derived class is type-compatible with all of its ancestor classes.
- As-a relationships
  - When public inheritance is inappropriate, (e.g., stack is not a list)
  - if your class needs access to the protected members of another class or
  - if you need to redefine methods in that class,
  - you can form an as-a relationship between your classes; that is, you can use private inheritance

```
class Stack : private List /* List's public and protected members, if
                                any, would be private members of Stack */
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Abstract Base Class

- Class that contains at least one pure virtual method.
- Is used only as the basis for derived classes and thus defines a minimum interface for its descendants.
- Has descendants but no instances.
- Should, in general, omit implementations except for the destructor and methods that provide access to private data members. That is, virtual methods in an abstract base class usually should be pure.
- Must implement any virtual method that is not pure, thereby providing a default implementation if a derived class chooses not to supply its own.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

C++ Interlude 4

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

C++ Interlude 4

© 2017 Pearson Education, Hoboken, NJ. All rights reserved